

Proyecto de Inteligencia Artificial: Razonamiento Espacial 3D y Localización con Gemini Robotics

1. Contexto y Objetivo del Experimento

En el ecosistema tradicional de ROS 2, la localización 3D suele depender de nubes de puntos (LiDAR) o sensores RGB-D calibrados. El objetivo de este experimento es utilizar el razonamiento espacial avanzado (Spatial Reasoning) del modelo **Gemini Robotics-ER 1.6 (gemini-robotics-er-1.6-preview)** para localizar la "caja de hamburguesa" en el espacio físico **3D**, combinando la inferencia de la IA con la geometría epipolar o proyecciones multi-vista.

Los estudiantes de IA deberán sustituir el pipeline clásico por una arquitectura donde el **Kinova Gen3** actúa como un agente activo que observa su entorno, utiliza a Gemini como su "cerebro visual" y calcula matemáticamente las coordenadas (X, Y, Z) para enviar el comando de *Grasp* a MoveIt 2.

2. ¿Por qué Gemini Robotics en lugar de un Object Detection clásico (ej. YOLO)?

Es una pregunta fundamental en IA robótica. Un modelo tradicional de detección de objetos (Object Detection) devuelve cuadros delimitadores basados en un dataset pre-entrenado, pero **carece de razonamiento**. Usar Gemini Robotics-ER 1.6 ofrece ventajas disruptivas para este proyecto:

- 1. Zero-Shot y Vocabulario Abierto:** Si usamos YOLO, tendríamos que recolectar y etiquetar miles de fotos de "cajas de hamburguesas" para entrenarlo. Gemini no requiere re-entrenamiento; simplemente le indicamos "caja de hamburguesa de cartón" en lenguaje natural y la encuentra. Mañana podemos cambiar a "vaso de refresco" y el sistema seguirá funcionando.
- 2. Razonamiento Contextual Espacial:** YOLO detectaría *todas* las cajas de la escena. Gemini permite prompts lógicos complejos: "*Encuentra la caja de hamburguesa que está apoyada sobre el carrito, ignora la que está en la basura*".
- 3. Entendimiento de Posibilidades (Affordances) y Seguridad:** Gemini puede evaluar el estado físico del objeto. Puedes preguntarle: "*Devuelve el Bounding Box de la caja, pero solo si la caja está cerrada y es seguro agarrarla desde arriba*". Un detector clásico no comprende la semántica de la seguridad física.
- 4. Agencia y Ejecución de Código:** Si la caja está muy lejos, Gemini puede autoejecutar código Python internamente para recortar la

imagen (crop), hacer zoom y recalcular una coordenada mucho más precisa. Es un agente, no solo una red neuronal *feed-forward*.

3. Razonamiento Espacial 3D en Gemini Robotics-ER 1.6

Investigaciones recientes de Google DeepMind revelan que la versión 1.6 de este modelo destaca en "**Multi-View Understanding**" y "**Embodied Reasoning**". Aunque la API base devuelve coordenadas normalizadas en 2D [Y, X], el modelo entiende profundamente la disposición 3D de la escena, proporciones físicas, restricciones de seguridad y oclusión.

Estrategias para alcanzar el 3D real: Existen dos enfoques que los estudiantes pueden implementar para obtener la coordenada (X,Y,Z) real:

1. **Inferencia Mono-Cámara + Depth (Enfoque Híbrido):** Gemini ubica semánticamente el objeto en 2D y el nodo de ROS extrae el valor Z del mapa de profundidad del tópico /camera/depth/...
 2. **Triangulación Multi-Vista Activa (Enfoque Avanzado):** El Kinova se mueve a dos poses distintas. Se envía a Gemini la "Vista A" y la "Vista B". Se obtienen dos puntos 2D [y_A, x_A] y [y_B, x_B]. Utilizando las matrices de transformación del robot (TF2) de ambas poses, se triangula matemáticamente la coordenada 3D exacta sin depender de un sensor infrarrojo.
-

4. Arquitectura del Nodo de Razonamiento Espacial

Se propone crear el nodo `gemini_3d_spatial_node`:

1. **Trigger Multi-Step:** El orquestador solicita localizar la "caja de hamburguesa de cartón".
 2. **Movimiento de Exploración:** El nodo comanda al brazo Kinova a ir a `observacion_pose_1`, captura la imagen y llama a la API.
 3. **Inferencia 2D (Gemini):**
 4. El modelo es llamado con `thinking_budget=0` (para agilidad) o con un presupuesto mayor si hay objetos apilados y se requiere razonamiento físico (ej. "señala la caja que esté encima y sea segura de agarrar").
 5. Retorna: [{"point": [Y, X], "label": "burger box"}]
 6. **Traducción Espacial (3D):**
 7. El nodo convierte [Y, X] normalizado a píxeles (u, v).
 8. Extrae el vector 3D combinando la matriz intrínseca de la cámara (`camera_info`) con la distancia de profundidad de la escena.
 9. **Publicación TF2:** Se hace *broadcast* del frame temporal estático `burger_box_ai_frame` anclado al `world` o `table_link`.
-

5. Prerrequisitos y Configuración del Entorno

Antes de escribir el nodo en ROS 2, los estudiantes deben configurar el entorno de ejecución para tener acceso a la API de Google:

1. **Obtener la API Key:** Ingresar a [Google AI Studio](#) y generar una clave de API. El uso de la capa gratuita es suficiente para este experimento.
 2. **Instalación del SDK Oficial:** Es crítico usar el nuevo SDK estándar de Google, no versiones depreciadas. Ejecutar en el entorno virtual de ROS 2: `bash pip install google-genai`
 3. **Configuración de la Variable de Entorno:** El SDK de google-genai busca automáticamente la clave en las variables del sistema. Para que el nodo de ROS 2 la encuentre, deben exportarla en la misma terminal donde ejecuten `ros2 run` o configurarla en su archivo `launch.py`: `bash export GEMINI_API_KEY="AIzaSyTuClaveSecretaAqui..."`
-

6. Guía de Implementación del Código Base

Fase 1: Llamada a la API de Gemini (Extracción Y, X)

Implementación sugerida para el servicio usando el SDK oficial de Google GenAI:

```
from google import genai
from google.genai import types

client = genai.Client()

prompt = """
Identify the cardboard burger box in the space.
Consider spatial relationships: it must be resting on the table, not held
Output ONLY a JSON array: [{"point": [y, x], "label": "burger box"}]
The points are normalized to 0-1000.
"""

response = client.models.generate_content(
    model="gemini-robotics-er-1.6-preview",
    contents=[
        types.Part.from_bytes(data=image_bytes, mime_type='image/jpeg'),
        prompt
    ],
    config=types.GenerateContentConfig(
        temperature=0.0,
        thinking_config=types.ThinkingConfig(thinking_budget=0)
    )
)

# ATENCIÓN: Gemini Robotics devuelve [Y, X].
# Para la geometría de cámara: u = X, v = Y.
```

```
# pixel_x = (x_normalized / 1000) * image_width
# pixel_y = (y_normalized / 1000) * image_height
```

Fase 2: Cálculo Espacial 3D (De-projection)

Para traducir la inferencia plana a volumen espacial, usando image_geometry:

```
from image_geometry import PinholeCameraModel

camera_model = PinholeCameraModel()
camera_model.fromCameraInfo(msg_camera_info)

u, v = pixel_x, pixel_y

# Opción A: Usar el sensor de Profundidad
z_depth = depth_image[int(v), int(u)]

# Proyección del rayo a 3D
ray = camera_model.projectPixelTo3dRay((u, v))
point_3d_camera_frame = (ray[0] * z_depth, ray[1] * z_depth, z_depth)
```

7. Criterios de Evaluación y Retos de IA (Rúbrica de 100 Puntos)

Competencia	Descripción del Reto	Puntaje
Razonamiento Contextual	El prompt debe ser capaz de discriminar entre una "caja vacía" y la "caja correcta de la orden" basándose en contexto espacial (ej. "la caja que está más cerca del carrito").	20%
Triangulación 3D o De-Projection	Cálculo matemático correcto del rayo 3D. El error de ubicación física (X,Y,Z) tras la transformación de TF2 no debe superar los 2.5 cm.	35%
Gestión de Oclusión y Seguridad	Uso de prompts para determinar si es seguro agarrar la caja (ej. "¿Hay obstáculos bloqueando la ruta superior de la caja?").	25%
Robustez Asíncrona	La llamada a la API debe correr en un hilo/ worker separado para no detener el spin() del nodo ROS 2, manejando timeouts de red con elegancia.	20%

8. Tips de Implementación Multi-Vista (Excelencia Académica)

Para los grupos que deseen prescindir del sensor de profundidad (simulando cámaras RGB económicas), pueden implementar el razonamiento multi-vista:

1. **Ejecución Activa:** El nodo comanda al brazo a moverse a 20 cm a la derecha de su pose actual.
2. **Stereo Artificial:** Llama a Gemini en ambas posiciones. Obtiene $(u1, v1)$ y $(u2, v2)$.
3. **Triangulación Directa:** Usando las matrices homogéneas de TF2 (camera_link_1 hacia base_link y camera_link_2 hacia base_link), se resuelven las ecuaciones de intersección de rayos para hallar la coordenada (X, Y, Z) absoluta. Esto demuestra una comprensión profunda del "Embodied AI" usando el robot como un sensor dinámico en el tiempo.